

```
In [1]: import pandas as pd
```

```
In [2]: import matplotlib.pyplot as plt
```

```
In [3]: import seaborn as sns
```

```
In [4]: from sklearn.model_selection import train_test_split
```

```
In [5]: from sklearn.preprocessing import LabelEncoder
```

```
In [6]: from sklearn.linear_model import LogisticRegression
```

```
In [7]: from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
In [8]: iris = pd.read_csv("IRIS (1).csv")
```

FileNotFoundError Traceback (most recent call last)

Cell In[8], line 1

```
----> 1 iris = pd.read_csv("IRIS (1).csv")
```

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:1026, in read_csv(filepath_or_buffer, sep, delimiter, header, names, index_col, usecols, dtype, engine, converters, true_values, false_values, skip_initialspace, skiprows, skipfooter, nrows, na_values, keep_default_na, na_filter, verbose, skip_blank_lines, parse_dates, infer_datetime_format, keep_date_col, date_parser, date_format, dayfirst, cache_dates, iterator, chunksize, compression, thousands, decimal, lineterminator, quotechar, quoting, doublequote, escapechar, comment, encoding, encoding_errors, dialect, on_bad_lines, delim_whitespace, low_memory, memory_map, float_precision, storage_options, dtype_backend)

```
1013 kwds_defaults = _refine_defaults_read(
1014     dialect,
1015     delimiter,
1016     (...)
1022     dtype_backend=dtype_backend,
1023 )
1024 kwds.update(kwds_defaults)
-> 1026 return _read(filepath_or_buffer, kwds)
```

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:620, in _read(filepath_or_buffer, kwds)

```
617 _validate_names(kwds.get("names", None))
619 # Create the parser.
--> 620 parser = TextFileReader(filepath_or_buffer, **kwds)
622 if chunksize or iterator:
623     return parser
```

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:1620, in TextFileReader.__init__(self, f, engine, **kwds)

```
1617 self.options["has_index_names"] = kwds["has_index_names"]
1619 self.handles: IOHandles | None = None
-> 1620 self._engine = self._make_engine(f, self.engine)
```

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:1880, in TextFileReader._make_engine(self, f, engine)

```
1878 if "b" not in mode:
1879     mode += "b"
-> 1880 self.handles = get_handle(
1881     f,
1882     mode,
1883     encoding=self.options.get("encoding", None),
1884     compression=self.options.get("compression", None),
1885     memory_map=self.options.get("memory_map", False),
1886     is_text=is_text,
1887     errors=self.options.get("encoding_errors", "strict"),
1888     storage_options=self.options.get("storage_options", None),
1889 )
1890 assert self.handles is not None
1891 f = self.handles.handle
```

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\io\common.py:873, in get_handle(path_or_buf, mode, encoding, compression, memory_map, is_text, errors, storage_options)

```
868 elif isinstance(handle, str):
869     # Check whether the filename is to be opened in binary mode.
870     # Binary mode does not support 'encoding' and 'newline'.
871     if ioargs.encoding and "b" not in ioargs.mode:
872         # Encoding
--> 873         handle = open(
874             handle,
875             ioargs.mode,
876             encoding=ioargs.encoding,
877             errors=errors,
878             newline="",
879         )
880     else:
881         # Binary mode
882         handle = open(handle, ioargs.mode)
```

FileNotFoundError: [Errno 2] No such file or directory: 'IRIS (1).csv'

```
In [9]: iris = pd.read_csv("C:\\Users\\neha gupta\\Downloads\\IRIS (1).csv")
```

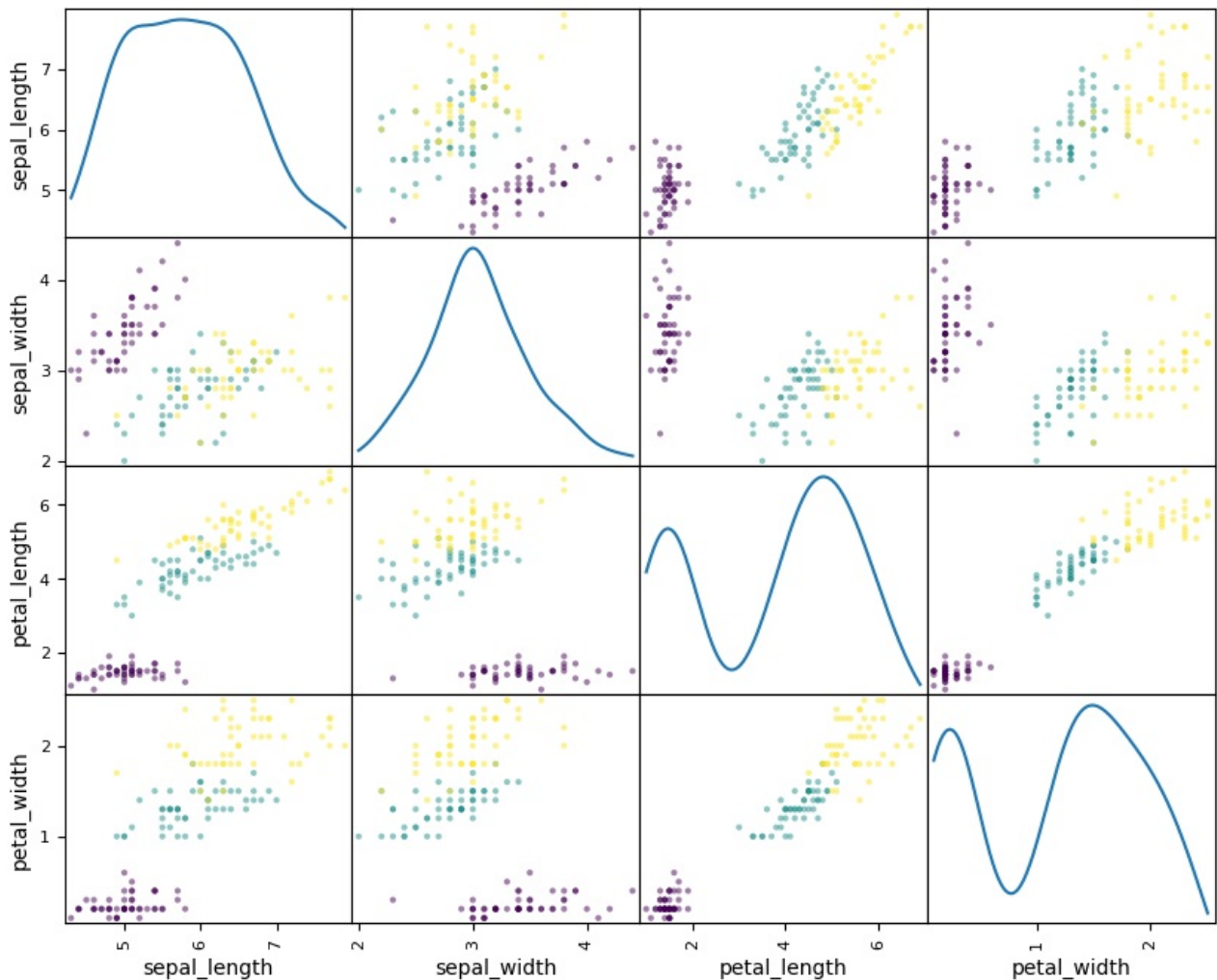
```
In [10]: iris.head()
```

```
Out[10]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
In [11]: pd.plotting.scatter_matrix(
    iris.drop("species", axis=1),
    figsize=(10, 8),
    diagonal="kde",
    c=pd.factorize(iris["species"])[0],
    cmap="viridis")
```

```
Out[11]: array([[<Axes: xlabel='sepal_length', ylabel='sepal_length'>,
  <Axes: xlabel='sepal_width', ylabel='sepal_length'>,
  <Axes: xlabel='petal_length', ylabel='sepal_length'>,
  <Axes: xlabel='petal_width', ylabel='sepal_length'>],
 [ <Axes: xlabel='sepal_length', ylabel='sepal_width'>,
  <Axes: xlabel='sepal_width', ylabel='sepal_width'>,
  <Axes: xlabel='petal_length', ylabel='sepal_width'>,
  <Axes: xlabel='petal_width', ylabel='sepal_width'>],
 [ <Axes: xlabel='sepal_length', ylabel='petal_length'>,
  <Axes: xlabel='sepal_width', ylabel='petal_length'>,
  <Axes: xlabel='petal_length', ylabel='petal_length'>,
  <Axes: xlabel='petal_width', ylabel='petal_length'>],
 [ <Axes: xlabel='sepal_length', ylabel='petal_width'>,
  <Axes: xlabel='sepal_width', ylabel='petal_width'>,
  <Axes: xlabel='petal_length', ylabel='petal_width'>,
  <Axes: xlabel='petal_width', ylabel='petal_width'>]], dtype=object)
```



```
In [12]: plt.suptitle("Scatter Matrix of Iris Dataset", y=1.02)
```

```
Out[12]: Text(0.5, 1.02, 'Scatter Matrix of Iris Dataset')
<Figure size 640x480 with 0 Axes>
```

```
In [13]: plt.show()
```

```

In [14]: X = iris.drop("species", axis=1)

In [15]: y = iris["species"]

In [16]: label_encoder = LabelEncoder()

In [17]: y_encoded = label_encoder.fit_transform(y)

In [18]: X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.2, random_state=42
)

In [19]: model = LogisticRegression(max_iter=200)

In [20]: model.fit(X_train, y_train)

Out[20]:
▼ LogisticRegression ⓘ ?
LogisticRegression(max_iter=200)

In [21]: y_pred = model.predict(X_test)

In [22]: accuracy = accuracy_score(y_test, y_pred)

In [23]: print("Model Accuracy:", accuracy)

```

Model Accuracy: 1.0

```

In [24]: print("\nClassification Report:\n", classification_report(y_test, y_pred, target_names=label_encoder.classes_))

```

```

Classification Report:
              precision    recall  f1-score   support

 Iris-setosa          1.00      1.00      1.00         10
 Iris-versicolor      1.00      1.00      1.00          9
 Iris-virginica       1.00      1.00      1.00         11

 accuracy              1.00          30
 macro avg             1.00      1.00      1.00          30
 weighted avg          1.00      1.00      1.00          30

```

```

In [25]: conf_matrix = confusion_matrix(y_test, y_pred)

```

```

In [26]: plt.figure(figsize=(6,4))

```

```

Out[26]: <Figure size 600x400 with 0 Axes>

```

<Figure size 600x400 with 0 Axes>

```

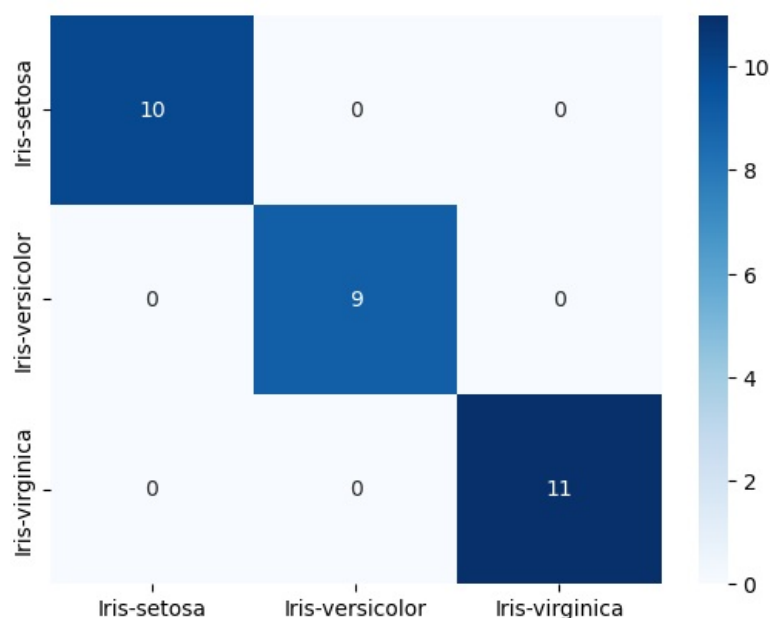
In [27]: sns.heatmap(
    conf_matrix, annot=True, cmap="Blues", fmt="d",
    xticklabels=label_encoder.classes_,
    yticklabels=label_encoder.classes_)

```

```

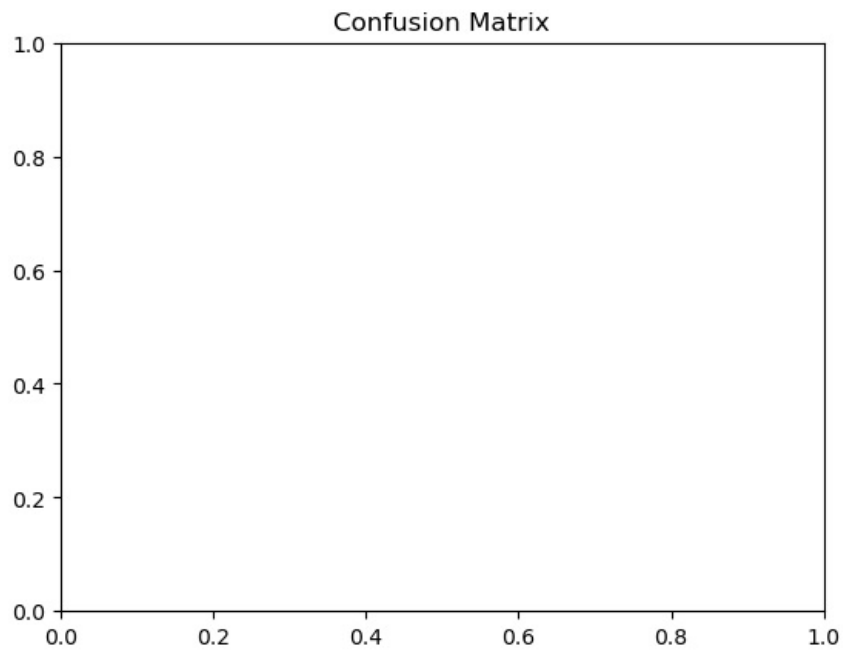
Out[27]: <Axes: >

```



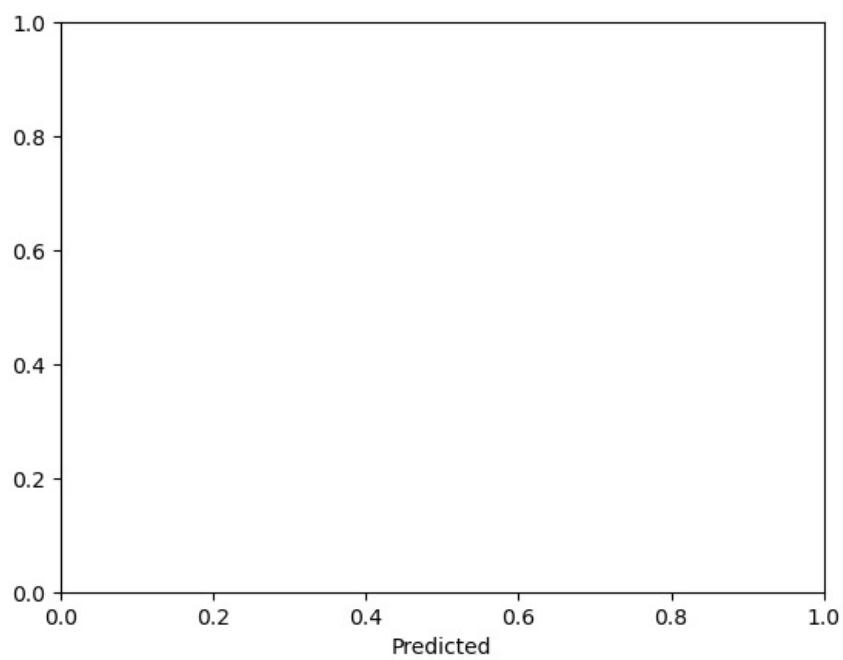
```
In [28]: plt.title("Confusion Matrix")
```

```
Out[28]: Text(0.5, 1.0, 'Confusion Matrix')
```



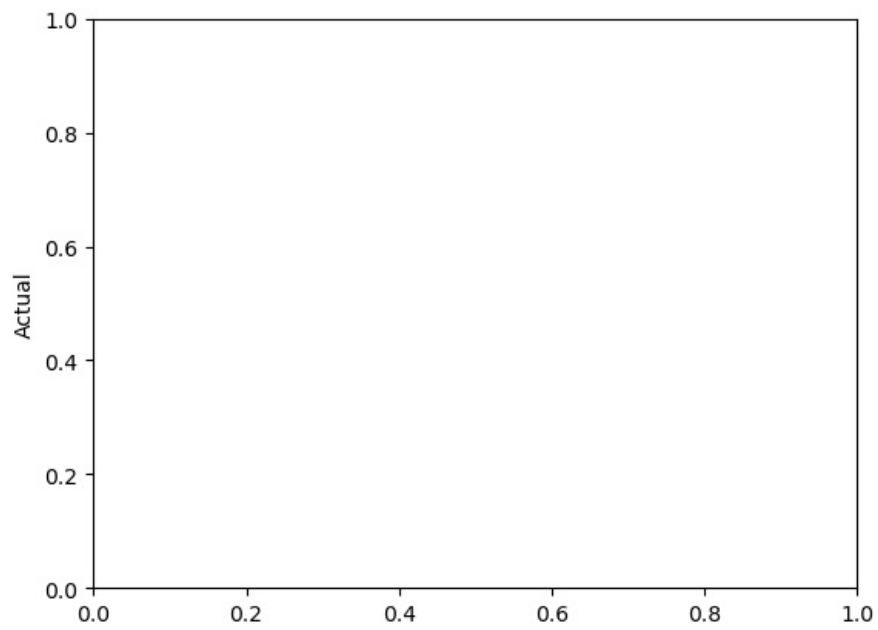
```
In [29]: plt.xlabel("Predicted")
```

```
Out[29]: Text(0.5, 0, 'Predicted')
```



```
In [30]: plt.ylabel("Actual")
```

```
Out[30]: Text(0, 0.5, 'Actual')
```



```
In [31]: plt.show()
```

```
In [ ]:
```